UNIVERSIDAD PRIVADA DEL NORTE / UAIN1288P

MedicRural-PE

Del registro de productos a una decisión clara: ¿qué hace falta reponer primero?

Java + Swing | **Orientado a objetos** | **Funcional + eventos**

- Jeremy Farfan
- German Flores
- Eduard Gozme
- Sebastian Huaman
- Gonzalo Portillo



Docente: Williams Alfonso Chullo Villagaray · 2026-II

Una lista no siempre muestra qué atender primero

Imaginemos un pequeño inventario con tres productos.

Guantes descartables
5 disponibles / mínimo 30
Atención
Faltan 25

Paracetamol
8 disponibles / mínimo 20
Atención
Faltan 12

Jeringas
16 disponibles / mínimo 10
A tiempo
Stock suficiente

La pregunta clave

¿Qué producto necesita atención y cuál tiene mayor faltante?

Necesitamos registrar, detectar críticos y ordenar las necesidades de reposición.

MedicRural-PE permite realizar siete funciones

Las agrupamos en tres tareas fáciles de recordar.

Registrar y gestionar

- **RF01** Registrar producto
- **RF02** Consultar inventario
- **RF03** Actualizar cantidad

Detectar y priorizar

- **RF04** Detectar stock crítico
- **RF05** Mostrar solo críticos
- **RF06** Ordenar por faltante

Resumir existencias

- **RF07** Ver un resumen

Productos registrados
Recursos críticos
Total de unidades

RF = Requerimiento Funcional = una acción que el programa debe poder realizar

Cada decisión responde a una necesidad

Lo que pedía el caso determinó cómo organizamos el software.

CÓDIGO	NECESIDAD O RESTRICCIÓN	DECISIÓN TOMADA
RF01 / RF03	**Registrar y actualizar**	**Inventario + RecursoFactory** Un lugar para guardar y crear registros
RF04–RF06	**Reconocer faltantes**	**esCritico() + AnalizadorInventario** Mostrar y ordenar lo que requiere atención
RD01	**Usar aplicación local**	**Java OpenJDK + Swing** Una ventana de escritorio con botones
RD02 / RD03	**Mantener alcance simple**	**Lista en memoria + datos ficticios** Sin base de datos ni registros de personas
RD04	**Facilitar cambios futuros**	**Separar interfaz, inventario y análisis** Cada parte tiene una responsabilidad

RF = requerimiento funcional · RD = restricción de diseño (una condición que decidimos respetar)

Tres formas de programar que se complementan

No son tres programas: son tres maneras de resolver partes del mismo sistema



Orientado a objetos

Cada producto tiene datos y reglas.

Medicamento e InsumoMedico comparten lo esencial de RecursoMedico.



Funcional

Analizamos una lista de recursos.

filter selecciona críticos; map y reduce ayudan a sumar cantidades.



Orientado a eventos

El usuario decide cuándo actuar.

Al hacer clic, Swing activa el método correspondiente.

Un clic → se consulta un objeto → se analiza la colección → se muestra el resultado

¿Cómo se comunican las partes del programa?



Singleton: un inventario compartido

Factory: un lugar para crear productos

Primero entendemos la regla; luego vemos Java

Ejemplo: si quedan 8 unidades y el mínimo es 20, el producto es crítico.

Paracetamol

8 de 20 unidades mínimas

CRÍTICO **Faltan 12 unidades**

Regla en RecursoMedico

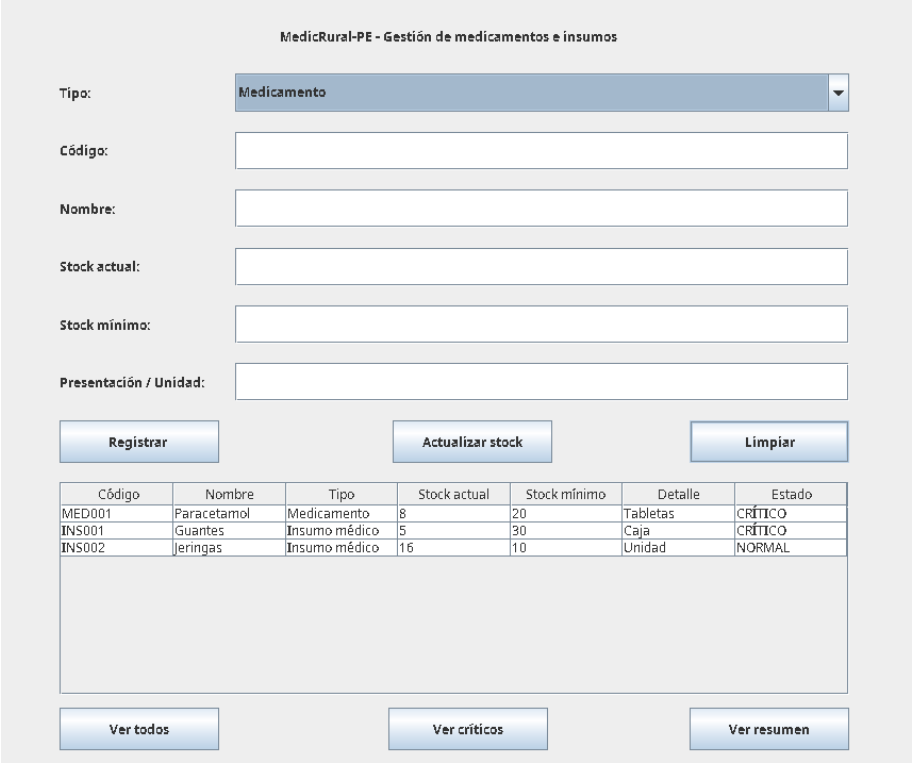
```
return stockActual <= stockMinimo;
```

Y para encontrar solo los críticos:

```
.filter(recurso -> recurso.esCritico())
```

¿Qué sucede cuando presionamos un botón?

Esta es la aplicación Java Swing ejecutándose con registros ficticios.



Captura real del prototipo · Datos ficticios

1 La persona hace clic

Por ejemplo: «Ver críticos».

2 Java ejecuta una acción

El botón activa su método.

3 El resultado aparece

La tabla muestra los recursos críticos.

Si un dato es inválido, aparece un aviso.

Tres pruebas automáticas respaldan funciones clave

JUnit compara un resultado real con el resultado que esperábamos.

Stock crítico

10 unidades ≤ mínimo 20

OK

Código duplicado

No permite registrar dos veces TEST001

OK

Prioridad de faltantes

25 unidades faltantes antes que 12

OK

3 / 3

casos aprobados

No significa que toda la aplicación esté probada.

¿Qué logramos y qué mejoraríamos?

Un prototipo pequeño, pero con decisiones de diseño que podemos explicar y probar

Lo que funciona

- Registrar y actualizar existencias
- Detectar y ordenar recursos críticos
- Analizar datos y responder a eventos

Lo que sigue

- Guardar información al cerrar el programa
- Ampliar las pruebas de interfaz
- Preparar controles para un uso real

Lo importante: podemos explicar cómo funciona, mostrarlo en vivo y reconocer sus límites.

FUENTES Y EVIDENCIAS

Referencias

Guía y rúbrica oficial	Universidad Privada del Norte. (2026). Desafío UAIN1288P [Documento del curso].
Código y pruebas reales	Equipo MedicRural-PE. (2026). MedicRural-PE [Código Java, UML y pruebas JUnit].
Informe del equipo	Equipo MedicRural-PE. (2026). Informe técnico final MedicRural-PE [Documento académico].

Repositorio del proyecto: github.com/Eduard-Gozme/MedicRuralPE